Internship report: Modularity in Interactive Theorem Provers

Arthur Adjedj (PhD Student)
Université Paris-Saclay, ENS Paris-Saclay

Interactive theorem provers (ITPs) do not easily allow users to adapt and extend existing codebases without either changing the original code or duplicating it. The pattern of extending definitions occurs particularly often in programming language theory, type theory, and program verification, leading to a lot of code duplication and high maintainance burden. Existing approaches to this problem either require encoding datatypes as complicated expressions, obfuscating the development, or rely on meta-programming facilities which were underdeveloped in most ITPs until recently.
We develop a principled way to produce modular code, leveraging the strengths of dependent types and modern meta-programming techniques.

Note. *This research topic has been collabaratively developed by Arthur Adjedj and Yannick Forster. Arthur Adjedj was supervised by Yannick Forster and hosted in the Cambium team at the Centre Inria de Paris during a five-month internship.*

Introduction

AA: TODO some (most ?) of this should be put in related works rather than the intro. AA: Emphasis to be put on the fact formalisations can be extended after the fact, see STLC case-study. Interactive Theorem Provers (ITPs), also known as proof assistants, are tools which allow for the development and mechanical verification of formal proofs. ITPs can be used to provide a very high level of guarantees for software (in particular the complete absence of bugs), and several projects make use of proof assistants to verify real-world software, such as the CompCert C compiler (Leroy [2009]), the sel4 operating systems kernel (Lyons et al. [2025]), and AWS' authorization policy language, Cedar (AWS [2026]). They have also been used successfully to formalise landmark theorems in mathematics such as the Four Colour Theorem or the Feit Thompson theorem, and the use of proof assistants is on the rise in mathematics departments, with big mathematical developments like Lean's Mathlib library (The mathlib Community [2020]) getting more and more public attention.

Like other programming languages, ITPs fall victim to the the fact that reusing definitions can be non-trivial. This is generally referred to as the **expression problem** (Wadler [1998]):

> "The goal is to define a datatype by cases, where one can add new cases to the datatype and new functions over the datatype, without recompiling existing code."

In fact, reusing and adapting existing data structures and programs modularly is a challenge for which few solutions have been produced over the years in industrial programming languages (Oliveira and Cook [2012]). It is even worse for ITPs, where in addition to programs, one needs to adapt proofs and dependently typed programs as well. Most projects that try to extend existing formalisation resort to copying the original content and adapting it for its own purpose. Since proofs about programs are arguably even harder to maintain than programs, this copy-paste approach incurs a huge maintenance burden.

A central contribution on matters of modularity in regular programming languages, "Data Types à la Carte" (Swierstra [2008]), provides a simple and elegant solution to the expression problem based on a parametrical approach to modular syntax in Haskell. This solution, however, cannot be easily adapted to ITPs since ITPs needs to ensure consistency via their type system. Instead, existing approaches either rely on complex encodings of datatypes that leak to the user, or on meta-programming.

"Meta-Theory à la Carte" (Delaware et al. [2013]) and "Pyrosome" (Jamner et al. [2025]) base their modularity on internal encodings of types (namely, impredicative church-encodings in the former, and Generalized Algebraic Theories in the latter). In both cases, the constructions are inneficient, incapable of extending previously user-defined inductive types (*i.e.,* expecting users to rely on the aforementionned encodings from the ground up), and expose the underlying internals of the encodings to the user. Having to deal with encodings of types rather than types adds a heavy burden in particular for new users interested in program verification. The lesson to include inductive datatypes natively has been learned early by popular ITPs (namely Rocq and Lean), who at first had no primitive notions of inductive types in their systems and then resorted to add those. Nowadays, most systems usually posess a syntactic notion of (co-)inductive types, and justify the ability to define these via some classes of models, allowing users to work with the abstractions these types provide, rather than with their encoding in said models. The only popular system that still relies on encodings to define such types, and manages to hide their implementation details well, is Isabelle/HOL.

On the other hand, Rocq à la Carte (Forster and Stark [2020]) relies on the meta-programming capabilities offered by the MetaRocq Project (Sozeau et al. [2020]) to allow users to construct new inductive types and functions by merging other inductive types, and/or adding new constructors. New functions on a "merged" datatype can then be constructed by merging past functions. The metaprogram then uses the given piece of information to reconstruct a new inductive type, and new functions, based on the informations given by the user. While great for extending constructions "vertically" (*i.e.,* by adding constructors to a type), this system does not allow for horizontal extensions (*i.e.,* extending the type signature of inductive types and their constructors). Furthermore, this approach has been hindered in the past by the lack of good metaprogramming frameworks in ITPs.

None of these systems, independently of whether they use encodings or the meta-programming, handle all of the type-system of ITPs they are implemented for (*e.g.,* none handle co-inductive types), or allow users to extend past formalisations "after the fact". Instead, formalisations have to be built from the ground up with the expectation that they will be extended with a specific framework in mind, making them much less useful for real-world uses.

Extensions

AA: TODO section intro AA: For each new thing presented, show the new syntax and how it can be used in practice. Build up an example over the sections using new tools progressively

Partial mappings

AA: TODO Add universes

$$
\begin{array}{rrr}
t, A, B, f & ::= & x & \text{(variable)} \\
& | & d & \text{(constant)} \\
& | & \_ & \text{(hole)} \\
& | & f\ t & \text{(application)} \\
& | & \lambda x : A.t & \text{(abstraction)} \\
& | & (x : A) \to B & \text{(}\Pi\text{-type)} \\
& | & \mathcal{U}_i & \text{(universe)}
\end{array}
$$

Judgement : $\Phi \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A'$ (In a mapping context Φ, term t of type A in context Γ maps to term t' of type A' in context Δ)

$$\frac{}{[\Phi \mid \Gamma \Rightarrow \Delta \vdash \mathcal{U}_i \Rightarrow \mathcal{U}_i : \mathcal{U}_{i+1} \Rightarrow \mathcal{U}_{i+1}]} \qquad \frac{[(x : A) \in \Gamma] \quad [(x : A') \in \Delta]}{[\Phi \mid \Gamma \Rightarrow \Delta \vdash x \Rightarrow x : A \Rightarrow A']} \qquad \frac{[(d \Rightarrow t : A \Rightarrow A') \in \Phi]}{[\Phi \mid \Gamma \Rightarrow \Delta \vdash d \Rightarrow t : A \Rightarrow A']}$$

$$\frac{[\Phi \mid \Gamma \Rightarrow \Delta \vdash f \Rightarrow f' : ((x : A) \to B) \Rightarrow ((x : A') \to B')] \quad [\Phi \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A']}{[\Phi \mid \Gamma \Rightarrow \Delta \vdash f\ t \Rightarrow f'\ t' : B[x := t] \Rightarrow B'[x := t']]}$$

$$\frac{[\Phi \mid (\Gamma, x : A) \Rightarrow (\Delta, x : A') \vdash t \Rightarrow t' : B \Rightarrow B']}{[\Phi \mid \Gamma \Rightarrow \Delta \vdash (\lambda x : A.t) \Rightarrow (\lambda x : A'.t') : ((x : A) \to B) \Rightarrow ((x : A') \to B')]}$$

$$\frac{[\Phi \mid (\Gamma, x : A) \Rightarrow (\Delta, x : A') \vdash t \Rightarrow t' : B \Rightarrow B']}{[\Phi \mid \Gamma \Rightarrow \Delta \vdash ((x : A) \to B) \Rightarrow ((x : A') \to B') : \mathcal{U}_i \Rightarrow \mathcal{U}_i]}$$

Inductive extensions

AA: Mention all the various auxiliary declarations that Lean uses internally and that need to be mapped appropriately

Definition extensions

Extending matchers

AA: Explain that the internal implementation of matchers does not reflect the kind of extension you would want to see in practice, mention auto-generalizations, and how they are actually handled in practice.

Making extensions modular

AA: TODO section intro

Inductive modularity

Definition modularity

AA: Empty for now, no work has been done here, write down your ideas

Case study: extending STLC

Related works

References

AWS (2026) Cedar Specification, 2026, https://github.com/cedar-policy/cedar-spec.

Delaware, B., S. Oliveira, B.C. d. and Schrijvers, T. (2013) Meta-theory à la carte, *SIGPLAN Not.* 2013;48(1): 207–218., doi:10.1145/2480359.2429094.

Forster, Y. and Stark, K. (2020) Coq à la carte: a practical approach to modular syntax with binders, In: Blanchette, J. and Hritcu, C. (eds.). *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020, New Orleans, LA, USA, January 20-21, 2020*, ACM 2020: pp. 186–200, doi:10.1145/3372885.3373817.

Jamner, D., Kammer, G., Nag, R. and Chlipala, A. (2025) Pyrosome: Verified Compilation for Modular Metatheory, *Proc ACM Program Lang.* 2025;9(OOPSLA2)., doi:10.1145/3763052.

Leroy, X. (2009) Formal verification of a realistic compiler, *Communications of the ACM.* 2009;52(7): 107–115., http://xavierleroy.org/publi/compcert-CACM.pdf.

Lyons, A., McLeod, K., Klein, G., et al. (2025) seL4/seL4: seL4 14.0.0, December 2025, doi:10.5281/zenodo.17904671.

The mathlib Community (2020) The Lean Mathematical Library, In. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP 2020, Association for Computing Machinery, New Orleans, LA, USA 2020: pp. 367–381, doi:10.1145/3372885.3373824.

Oliveira, B.C.d.S. and Cook, W.R. (2012) Extensibility for the Masses, In: Noble, J. (ed.). *ECOOP 2012 – Object-Oriented Programming*, Springer Berlin Heidelberg, Berlin, Heidelberg 2012: pp. 2–27.

Sozeau, M., Anand, A., Boulier, S., et al. (2020) The MetaCoq Project, *Journal of Automated Reasoning*. advance online publication 2020., doi:10.1007/s10817-019-09540-0.

Swierstra, W. (2008) Data types à la carte, *J Funct Program*. 2008;18(4): 423–436., doi:10.1017/S0956796808006758.

Wadler, P. (1998) The expression problem, November 1998, https://homepages.inf.ed.ac.uk/wadler/papers/expression/expression.txt.